PPTX creation, editing, and analysis

Overview

A user may ask you to create, edit, or analyze the contents of a .pptx file. A .pptx file is essentially a ZIP

Reading and analyzing content

Text extraction
If you just need to read the text contents of a presentation, you should convert the document to markdown:

```bash
# Convert document to markdown
python -m markitdown path-to-file.pptx
```

Raw XML access
You need raw XML access for: comments, speaker notes, slide layouts, animations, design elements, and complex

Unpacking a file
`python ooxml/scripts/unpack.py <office_file> <output_dir>`

Note: The unpack.py script is located at `skills/pptx/ooxml/scripts/unpack.py` relative to the project roo

Key file structures
* `ppt/presentation.xml` - Main presentation metadata and slide references
* `ppt/slides/slide{N}.xml` - Individual slide contents (slide1.xml, slide2.xml, etc.)
* `ppt/notesSlides/notesSlide{N}.xml` - Speaker notes for each slide
* `ppt/comments/modernComment_*.xml` - Comments for specific slides
* `ppt/slideLayouts/` - Layout templates for slides
* `ppt/slideMasters/` - Master slide templates
* `ppt/theme/` - Theme and styling information
* `ppt/media/` - Images and other media files

Typography and color extraction
When given an example design to emulate: Always analyze the presentation's typography and colors first usi
1. **Read theme file**: Check `ppt/theme/theme1.xml` for colors (`<a:clrScheme>`) and fonts (`<a:fontScheme>`)
2. **Sample slide content**: Examine `ppt/slides/slide1.xml` for actual font usage (`<a:rPr>`) and colors
3. **Search for patterns**: Use grep to find color (`<a:solidFill>`, `<a:srgbClr>`) and font references across

Creating a new PowerPoint presentation **without a template**

When creating a new PowerPoint presentation from scratch, use the **html2pptx** workflow to convert HTML slide

Design Principles

CRITICAL: Before creating any presentation, analyze the content and choose appropriate design elements:
1. **Consider the subject matter**: What is this presentation about? What tone, industry, or mood does it sugg
2. **Check for branding**: If the user mentions a company/organization, consider their brand colors and identi
3. **Match palette to content**: Select colors that reflect the subject
4. **State your approach**: Explain your design choices before writing code

Requirements:
- ■ State your content-informed design approach BEFORE writing code
- ■ Use web-safe fonts only: Arial, Helvetica, Times New Roman, Georgia, Courier New, Verdana, Tahoma, Trebuch
- ■ Create clear visual hierarchy through size, weight, and color
- ■ Ensure readability: strong contrast, appropriately sized text, clean alignment
- ■ Be consistent: repeat patterns, spacing, and visual language across slides

Color Palette Selection

Choosing colors creatively:
- **Think beyond defaults**: What colors genuinely match this specific topic? Avoid autopilot choices.
- **Consider multiple angles**: Topic, industry, mood, energy level, target audience, brand identity (if menti
- **Be adventurous**: Try unexpected combinations - a healthcare presentation doesn't have to be green, financ
- **Build your palette**: Pick 3-5 colors that work together (dominant colors + supporting tones + accent)
- **Ensure contrast**: Text must be clearly readable on backgrounds

Example color palettes (use these to spark creativity - choose one, adapt it, or create your own):

1. **Classic Blue**: Deep navy (#1C2833), slate gray (#2E4053), silver (#AAB7B8), off-white (#F4F6F6)
2. **Teal & Coral**: Teal (#5EA8A7), deep teal (#277884), coral (#FE4447), white (#FFFFFF)
3. **Bold Red**: Red (#C0392B), bright red (#E74C3C), orange (#F39C12), yellow (#F1C40F), green (#2ECC71)
4. **Warm Blush**: Mauve (#A49393), blush (#EED6D3), rose (#E8B4B8), cream (#FAF7F2)
5. **Burgundy Luxury**: Burgundy (#5D1D2E), crimson (#951233), rust (#C15937), gold (#997929)
6. **Deep Purple & Emerald**: Purple (#B165FB), dark blue (#181B24), emerald (#40695B), white (#FFFFFF)
7. **Cream & Forest Green**: Cream (#FFE1C7), forest green (#40695B), white (#FCFCFC)
8. **Pink & Purple**: Pink (#F8275B), coral (#FF574A), rose (#FF737D), purple (#3D2F68)
9. **Lime & Plum**: Lime (#C5DE82), plum (#7C3A5F), coral (#FD8C6E), blue-gray (#98ACB5)
10. **Black & Gold**: Gold (#BF9A4A), black (#000000), cream (#F4F6F6)
11. **Sage & Terracotta**: Sage (#87A96B), terracotta (#E07A5F), cream (#F4F1DE), charcoal (#2C2C2C)
12. **Charcoal & Red**: Charcoal (#292929), red (#E33737), light gray (#CCCBCB)
13. **Vibrant Orange**: Orange (#F96D00), light gray (#F2F2F2), charcoal (#222831)
14. **Forest Green**: Black (#191A19), green (#4E9F3D), dark green (#1E5128), white (#FFFFFF)
15. **Retro Rainbow**: Purple (#722880), pink (#D72D51), orange (#EB5C18), amber (#F08800), gold (#DEB600)
16. **Vintage Earthy**: Mustard (#E3B448), sage (#CBD18F), forest green (#3A6B35), cream (#F4F1DE)
17. **Coastal Rose**: Old rose (#AD7670), beaver (#B49886), eggshell (#F3ECDC), ash gray (#BFD5BE)
18. **Orange & Turquoise**: Light orange (#FC993E), grayish turquoise (#667C6F), white (#FCFCFC)

Visual Details Options

Geometric Patterns:
- Diagonal section dividers instead of horizontal
- Asymmetric column widths (30/70, 40/60, 25/75)
- Rotated text headers at 90° or 270°
- Circular/hexagonal frames for images
- Triangular accent shapes in corners
- Overlapping shapes for depth

Border & Frame Treatments:
- Thick single-color borders (10-20pt) on one side only
- Double-line borders with contrasting colors
- Corner brackets instead of full frames
- L-shaped borders (top+left or bottom+right)
- Underline accents beneath headers (3-5pt thick)

Typography Treatments:
- Extreme size contrast (72pt headlines vs 11pt body)
- All-caps headers with wide letter spacing
- Numbered sections in oversized display type
- Monospace (Courier New) for data/stats/technical content
- Condensed fonts (Arial Narrow) for dense information
- Outlined text for emphasis

Chart & Data Styling:
- Monochrome charts with single accent color for key data
- Horizontal bar charts instead of vertical
- Dot plots instead of bar charts
- Minimal gridlines or none at all
- Data labels directly on elements (no legends)
- Oversized numbers for key metrics

Layout Innovations:

- Full-bleed images with text overlays
- Sidebar column (20-30% width) for navigation/context
- Modular grid systems (3×3, 4×4 blocks)
- Z-pattern or F-pattern content flow
- Floating text boxes over colored shapes
- Magazine-style multi-column layouts

Background Treatments:
- Solid color blocks occupying 40-60% of slide
- Gradient fills (vertical or diagonal only)
- Split backgrounds (two colors, diagonal or vertical)
- Edge-to-edge color bands
- Negative space as a design element

Layout Tips
When creating slides with charts or tables:
- **Two-column layout (PREFERRED)**: Use a header spanning the full width, then two columns below - text/bulle
- **Full-slide layout**: Let the featured content (chart/table) take up the entire slide for maximum impact an
- **NEVER vertically stack**: Do not place charts/tables below text in a single column - this causes poor read

Workflow
1. **MANDATORY - READ ENTIRE FILE**: Read [`html2pptx.md`](html2pptx.md) completely from start to finish. **NE
2. Create an HTML file for each slide with proper dimensions (e.g., 720pt × 405pt for 16:9)
   - Use `<p>`, `<h1>`-`<h6>`, `<ul>`, `<ol>` for all text content
   - Use `class="placeholder"` for areas where charts/tables will be added (render with gray background for vi
   - **CRITICAL**: Rasterize gradients and icons as PNG images FIRST using Sharp, then reference in HTML
   - **LAYOUT**: For slides with charts/tables/images, use either full-slide layout or two-column layout for b
3. Create and run a JavaScript file using the [`html2pptx.js`](scripts/html2pptx.js) library to convert HTML s
   - Use the `html2pptx()` function to process each HTML file
   - Add charts and tables to placeholder areas using PptxGenJS API
   - Save the presentation using `pptx.writeFile()`
4. **Visual validation**: Generate thumbnails and inspect for layout issues
   - Create thumbnail grid: `python scripts/thumbnail.py output.pptx workspace/thumbnails --cols 4`
   - Read and carefully examine the thumbnail image for:
     - **Text cutoff**: Text being cut off by header bars, shapes, or slide edges
     - **Text overlap**: Text overlapping with other text or shapes
     - **Positioning issues**: Content too close to slide boundaries or other elements
     - **Contrast issues**: Insufficient contrast between text and backgrounds
   - If issues found, adjust HTML margins/spacing/colors and regenerate the presentation
   - Repeat until all slides are visually correct

Editing an existing PowerPoint presentation

When edit slides in an existing PowerPoint presentation, you need to work with the raw Office Open XML (OOXML)

Workflow
1. **MANDATORY - READ ENTIRE FILE**: Read [`ooxml.md`](ooxml.md) (~500 lines) completely from start to finish.
2. Unpack the presentation: `python ooxml/scripts/unpack.py <office_file> <output_dir>`
3. Edit the XML files (primarily `ppt/slides/slide{N}.xml` and related files)
4. **CRITICAL**: Validate immediately after each edit and fix any validation errors before proceeding: `python
5. Pack the final presentation: `python ooxml/scripts/pack.py <input_directory> <office_file>`

Creating a new PowerPoint presentation **using a template**

When you need to create a presentation that follows an existing template's design, you'll need to duplicate an

Workflow
1. **Extract template text AND create visual thumbnail grid**:
   * Extract text: `python -m markitdown template.pptx > template-content.md`
   * Read `template-content.md`: Read the entire file to understand the contents of the template presentation.
   * Create thumbnail grids: `python scripts/thumbnail.py template.pptx`
   * See [Creating Thumbnail Grids](#creating-thumbnail-grids) section for more details

2. **Analyze template and save inventory to a file**:
   * **Visual Analysis**: Review thumbnail grid(s) to understand slide layouts, design patterns, and visual st
   * Create and save a template inventory file at `template-inventory.md` containing:
     ```markdown
     # Template Inventory Analysis
     **Total Slides: [count]**
     **IMPORTANT: Slides are 0-indexed (first slide = 0, last slide = count-1)**

     ## [Category Name]
     - Slide 0: [Layout code if available] - Description/purpose
     - Slide 1: [Layout code] - Description/purpose
     - Slide 2: [Layout code] - Description/purpose
     [... EVERY slide must be listed individually with its index ...]
     ```
   * **Using the thumbnail grid**: Reference the visual thumbnails to identify:
     - Layout patterns (title slides, content layouts, section dividers)
     - Image placeholder locations and counts
     - Design consistency across slide groups
     - Visual hierarchy and structure
   * This inventory file is REQUIRED for selecting appropriate templates in the next step

3. **Create presentation outline based on template inventory**:
   * Review available templates from step 2.
   * Choose an intro or title template for the first slide. This should be one of the first templates.
   * Choose safe, text-based layouts for the other slides.
   * **CRITICAL: Match layout structure to actual content**:
     - Single-column layouts: Use for unified narrative or single topic
     - Two-column layouts: Use ONLY when you have exactly 2 distinct items/concepts
     - Three-column layouts: Use ONLY when you have exactly 3 distinct items/concepts
     - Image + text layouts: Use ONLY when you have actual images to insert
     - Quote layouts: Use ONLY for actual quotes from people (with attribution), never for emphasis
     - Never use layouts with more placeholders than you have content
     - If you have 2 items, don't force them into a 3-column layout
     - If you have 4+ items, consider breaking into multiple slides or using a list format
   * Count your actual content pieces BEFORE selecting the layout
   * Verify each placeholder in the chosen layout will be filled with meaningful content
   * Select one option representing the **best** layout for each content section.
   * Save `outline.md` with content AND template mapping that leverages available designs
   * Example template mapping:
     ```
     # Template slides to use (0-based indexing)
     # WARNING: Verify indices are within range! Template with 73 slides has indices 0-72
     # Mapping: slide numbers from outline -> template slide indices
     template_mapping = [
         0,   # Use slide 0 (Title/Cover)
         34,  # Use slide 34 (B1: Title and body)
         34,  # Use slide 34 again (duplicate for second B1)
         50,  # Use slide 50 (E1: Quote)
         54,  # Use slide 54 (F2: Closing + Text)
     ]
     ```

4. **Duplicate, reorder, and delete slides using `rearrange.py`**:
   * Use the `scripts/rearrange.py` script to create a new presentation with slides in the desired order:
     ```bash
     python scripts/rearrange.py template.pptx working.pptx 0,34,34,50,52
     ```
   * The script handles duplicating repeated slides, deleting unused slides, and reordering automatically
   * Slide indices are 0-based (first slide is 0, second is 1, etc.)
   * The same slide index can appear multiple times to duplicate that slide

5. **Extract ALL text using the `inventory.py` script**:
   * **Run inventory extraction**:
     ```bash
     python scripts/inventory.py working.pptx text-inventory.json
     ```
   * **Read text-inventory.json**: Read the entire text-inventory.json file to understand all shapes and their

   * The inventory JSON structure:
     ```json
     {
       "slide-0": {
         "shape-0": {
           "placeholder_type": "TITLE",  // or null for non-placeholders
           "left": 1.5,                  // position in inches
           "top": 2.0,
           "width": 7.5,
           "height": 1.2,
           "paragraphs": [
             {
               "text": "Paragraph text",
               // Optional properties (only included when non-default):
               "bullet": true,           // explicit bullet detected
               "level": 0,               // only included when bullet is true
               "alignment": "CENTER",    // CENTER, RIGHT (not LEFT)
               "space_before": 10.0,     // space before paragraph in points
               "space_after": 6.0,       // space after paragraph in points
               "line_spacing": 22.4,     // line spacing in points
               "font_name": "Arial",     // from first run
               "font_size": 14.0,        // in points
               "bold": true,
               "italic": false,
               "underline": false,
               "color": "FF0000"         // RGB color
             }
           ]
         }
       }
     }
     ```

   * Key features:
     - **Slides**: Named as "slide-0", "slide-1", etc.
     - **Shapes**: Ordered by visual position (top-to-bottom, left-to-right) as "shape-0", "shape-1", etc.
     - **Placeholder types**: TITLE, CENTER_TITLE, SUBTITLE, BODY, OBJECT, or null
     - **Default font size**: `default_font_size` in points extracted from layout placeholders (when available
     - **Slide numbers are filtered**: Shapes with SLIDE_NUMBER placeholder type are automatically excluded fr
     - **Bullets**: When `bullet: true`, `level` is always included (even if 0)
     - **Spacing**: `space_before`, `space_after`, and `line_spacing` in points (only included when set)
     - **Colors**: `color` for RGB (e.g., "FF0000"), `theme_color` for theme colors (e.g., "DARK_1")
     - **Properties**: Only non-default values are included in the output

6. **Generate replacement text and save the data to a JSON file**
   Based on the text inventory from the previous step:
   - **CRITICAL**: First verify which shapes exist in the inventory - only reference shapes that are actually
   - **VALIDATION**: The replace.py script will validate that all shapes in your replacement JSON exist in the
     - If you reference a non-existent shape, you'll get an error showing available shapes
     - If you reference a non-existent slide, you'll get an error indicating the slide doesn't exist
     - All validation errors are shown at once before the script exits
   - **IMPORTANT**: The replace.py script uses inventory.py internally to identify ALL text shapes
   - **AUTOMATIC CLEARING**: ALL text shapes from the inventory will be cleared unless you provide "paragraphs
   - Add a "paragraphs" field to shapes that need content (not "replacement_paragraphs")
   - Shapes without "paragraphs" in the replacement JSON will have their text cleared automatically

- Paragraphs with bullets will be automatically left aligned. Don't set the `alignment` property on when `"
- Generate appropriate replacement content for placeholder text
- Use shape size to determine appropriate content length
- **CRITICAL**: Include paragraph properties from the original inventory - don't just provide text
- **IMPORTANT**: When bullet: true, do NOT include bullet symbols (•, -, *) in text - they're added automat
- **ESSENTIAL FORMATTING RULES**:
  - Headers/titles should typically have `"bold": true`
  - List items should have `"bullet": true, "level": 0` (level is required when bullet is true)
  - Preserve any alignment properties (e.g., `"alignment": "CENTER"` for centered text)
  - Include font properties when different from default (e.g., `"font_size": 14.0`, `"font_name": "Lora"`)
  - Colors: Use `"color": "FF0000"` for RGB or `"theme_color": "DARK_1"` for theme colors
  - The replacement script expects **properly formatted paragraphs**, not just text strings
  - **Overlapping shapes**: Prefer shapes with larger default_font_size or more appropriate placeholder_typ
- Save the updated inventory with replacements to `replacement-text.json`
- **WARNING**: Different template layouts have different shape counts - always check the actual inventory b

Example paragraphs field showing proper formatting:
```json
"paragraphs": [
  {
    "text": "New presentation title text",
    "alignment": "CENTER",
    "bold": true
  },
  {
    "text": "Section Header",
    "bold": true
  },
  {
    "text": "First bullet point without bullet symbol",
    "bullet": true,
    "level": 0
  },
  {
    "text": "Red colored text",
    "color": "FF0000"
  },
  {
    "text": "Theme colored text",
    "theme_color": "DARK_1"
  },
  {
    "text": "Regular paragraph text without special formatting"
  }
]
```

Shapes not listed in the replacement JSON are automatically cleared:
```json
{
  "slide-0": {
    "shape-0": {
      "paragraphs": [...] // This shape gets new text
    }
    // shape-1 and shape-2 from inventory will be cleared automatically
  }
}
```

Common formatting patterns for presentations:
- Title slides: Bold text, sometimes centered
- Section headers within slides: Bold text

   - Bullet lists: Each item needs `"bullet": true, "level": 0`
   - Body text: Usually no special properties needed
   - Quotes: May have special alignment or font properties

7. **Apply replacements using the `replace.py` script**
   ```bash
   python scripts/replace.py working.pptx replacement-text.json output.pptx
   ```

   The script will:
   - First extract the inventory of ALL text shapes using functions from inventory.py
   - Validate that all shapes in the replacement JSON exist in the inventory
   - Clear text from ALL shapes identified in the inventory
   - Apply new text only to shapes with "paragraphs" defined in the replacement JSON
   - Preserve formatting by applying paragraph properties from the JSON
   - Handle bullets, alignment, font properties, and colors automatically
   - Save the updated presentation

   Example validation errors:
   ```
   ERROR: Invalid shapes in replacement JSON:
     - Shape 'shape-99' not found on 'slide-0'. Available shapes: shape-0, shape-1, shape-4
     - Slide 'slide-999' not found in inventory
   ```

   ```
   ERROR: Replacement text made overflow worse in these shapes:
     - slide-0/shape-2: overflow worsened by 1.25" (was 0.00", now 1.25")
   ```

Creating Thumbnail Grids

To create visual thumbnail grids of PowerPoint slides for quick analysis and reference:

```bash
python scripts/thumbnail.py template.pptx [output_prefix]
```

Features:
- Creates: `thumbnails.jpg` (or `thumbnails-1.jpg`, `thumbnails-2.jpg`, etc. for large decks)
- Default: 5 columns, max 30 slides per grid (5×6)
- Custom prefix: `python scripts/thumbnail.py template.pptx my-grid`
  - Note: The output prefix should include the path if you want output in a specific directory (e.g., `workspa
- Adjust columns: `--cols 4` (range: 3-6, affects slides per grid)
- Grid limits: 3 cols = 12 slides/grid, 4 cols = 20, 5 cols = 30, 6 cols = 42
- Slides are zero-indexed (Slide 0, Slide 1, etc.)

Use cases:
- Template analysis: Quickly understand slide layouts and design patterns
- Content review: Visual overview of entire presentation
- Navigation reference: Find specific slides by their visual appearance
- Quality check: Verify all slides are properly formatted

Examples:
```bash
# Basic usage
python scripts/thumbnail.py presentation.pptx

# Combine options: custom name, columns
python scripts/thumbnail.py template.pptx analysis --cols 4
```

Converting Slides to Images

To visually analyze PowerPoint slides, convert them to images using a two-step process:

1. **Convert PPTX to PDF**:
   ```bash
   soffice --headless --convert-to pdf template.pptx
   ```

2. **Convert PDF pages to JPEG images**:
   ```bash
   pdftoppm -jpeg -r 150 template.pdf slide
   ```
   This creates files like `slide-1.jpg`, `slide-2.jpg`, etc.

Options:
- `-r 150`: Sets resolution to 150 DPI (adjust for quality/size balance)
- `-jpeg`: Output JPEG format (use `-png` for PNG if preferred)
- `-f N`: First page to convert (e.g., `-f 2` starts from page 2)
- `-l N`: Last page to convert (e.g., `-l 5` stops at page 5)
- `slide`: Prefix for output files

Example for specific range:
```bash
pdftoppm -jpeg -r 150 -f 2 -l 5 template.pdf slide  # Converts only pages 2-5
```

Code Style Guidelines
IMPORTANT: When generating code for PPTX operations:
- Write concise code
- Avoid verbose variable names and redundant operations
- Avoid unnecessary print statements

Dependencies

Required dependencies (should already be installed):

- **markitdown**: `pip install "markitdown[pptx]"` (for text extraction from presentations)
- **pptxgenjs**: `npm install -g pptxgenjs` (for creating presentations via html2pptx)
- **playwright**: `npm install -g playwright` (for HTML rendering in html2pptx)
- **react-icons**: `npm install -g react-icons react react-dom` (for icons)
- **sharp**: `npm install -g sharp` (for SVG rasterization and image processing)
- **LibreOffice**: `sudo apt-get install libreoffice` (for PDF conversion)
- **Poppler**: `sudo apt-get install poppler-utils` (for pdftoppm to convert PDF to images)
- **defusedxml**: `pip install defusedxml` (for secure XML parsing)